Advanced Dependencies

Parameterized dependencies

All the dependencies we have seen are a fixed function or class.

But there could be cases where you want to be able to set parameters on the dependency, without having to declare many different functions or classes.

Let's imagine that we want to have a dependency that checks if the query parameter `q` contains some fixed content.

But we want to be able to parameterize that fixed content.

A "callable" instance

In Python there's a way to make an instance of a class a "callable".

Not the class itself (which is already a callable), but an instance of that class.

To do that, we declare a method `__call__`:

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: Annotated[bool, Depends(checker)]):
    return {"fixed_content_in_query": fixed_content_included}
```

Ask AI

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: bool = Depends(checker)):
    return {"fixed_content_in_query": fixed_content_included}
```

In this case, this `__call__` is what **FastAPI** will use to check for additional parameters and sub-dependencies, and this is what will be called to pass a value to the parameter in your *path operation function* later.

Parameterize the instance

And now, we can use `__init__` to declare the parameters of the instance that we can use to "parameterize" the dependency:

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: Annotated[bool, Depends(checker)]):
```

```
    return {"fixed_content_in_query": fixed_content_included}
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: bool = Depends(checker)):
    return {"fixed_content_in_query": fixed_content_included}
```

In this case, **FastAPI** won't ever touch or care about `__init__`, we will use it directly in our code.

Create an instance

We could create an instance of this class with:

Python 3.10+

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
```

```
async def read_query_check(fixed_content_included: Annotated[bool, Depends(checker)]):
    return {"fixed_content_in_query": fixed_content_included}
```

Other versions and variants

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: bool = Depends(checker)):
    return {"fixed_content_in_query": fixed_content_included}
```

And that way we are able to "parameterize" our dependency, that now has `"bar"` inside of it, as the attribute `checker.fixed_content`.

Use the instance as a dependency

Then, we could use this `checker` in a `Depends(checker)`, instead of `Depends(FixedContentQueryChecker)`, because the dependency is the instance, `checker`, not the class itself.

And when solving the dependency, **FastAPI** will call this `checker` like:

```
checker(q="somequery")
```

...and pass whatever that returns as the value of the dependency in our *path operation function* as the parameter `fixed_content_included`:

Python 3.10+

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
```

```
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: Annotated[bool, Depends(checker)]):
    return {"fixed_content_in_query": fixed_content_included}
```

Other versions and variants

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):
        self.fixed_content = fixed_content

    def __call__(self, q: str = ""):
        if q:
            return self.fixed_content in q
        return False


checker = FixedContentQueryChecker("bar")


@app.get("/query-checker/")
async def read_query_check(fixed_content_included: bool = Depends(checker)):
    return {"fixed_content_in_query": fixed_content_included}
```

Tip

All this might seem contrived. And it might not be very clear how is it useful yet.

These examples are intentionally simple, but show how it all works.

In the chapters about security, there are utility functions that are implemented in this same way.

If you understood all this, you already know how those utility tools for security work underneath.

Dependencies with `yield`, `HTTPException`, `except` and Background Tasks

> **Warning**
>
> You most probably don't need these technical details.
>
> These details are useful mainly if you had a FastAPI application older than `0.121.0` and you are facing issues with dependencies with `yield`.

Dependencies with `yield` have evolved over time to account for the different use cases and to fix some issues, here's a summary of what has changed.

Dependencies with `yield` and `scope`

In version `0.121.0`, FastAPI added support for `Depends(scope="function")` for dependencies with `yield`.

Using `Depends(scope="function")`, the exit code after `yield` is executed right after the *path operation function* is finished, before the response is sent back to the client.

And when using `Depends(scope="request")` (the default), the exit code after `yield` is executed after the response is sent.

You can read more about it in the docs for Dependencies with `yield` - Early exit and `scope`.

Dependencies with `yield` and `StreamingResponse`, Technical Details

Before FastAPI `0.118.0`, if you used a dependency with `yield`, it would run the exit code after the *path operation function* returned but right before sending the response.

The intention was to avoid holding resources for longer than necessary, waiting for the response to travel through the network.

This change also meant that if you returned a `StreamingResponse`, the exit code of the dependency with `yield` would have been already run.

For example, if you had a database session in a dependency with `yield`, the `StreamingResponse` would not be able to use that session while streaming data because the session would have already been closed in the exit code after `yield`.

This behavior was reverted in `0.118.0`, to make the exit code after `yield` be executed after the response is sent.

> **Info**
>
> As you will see below, this is very similar to the behavior before version `0.106.0`, but with several improvements and bug fixes for corner cases.

Use Cases with Early Exit Code

There are some use cases with specific conditions that could benefit from the old behavior of running the exit code of dependencies with `yield` before sending the response.

For example, imagine you have code that uses a database session in a dependency with `yield` only to verify a user, but the database session is never used again in the *path operation function*, only in the dependency, **and** the response takes a long time to be sent, like a `StreamingResponse` that sends data slowly, but for some reason doesn't use the database.

In this case, the database session would be held until the response is finished being sent, but if you don't use it, then it wouldn't be necessary to hold it.

Here's how it could look like:

Python 3.10+

```python
import time
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from sqlmodel import Field, Session, SQLModel, create_engine

engine = create_engine("postgresql+psycopg://postgres:postgres@localhost/db")


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str


app = FastAPI()


def get_session():
    with Session(engine) as session:
        yield session


def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")


def generate_stream(query: str):
    for ch in query:
        yield ch
        time.sleep(0.1)


@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

The exit code, the automatic closing of the `Session` in:

Python 3.10+

```python
# Code above omitted 👆

def get_session():
    with Session(engine) as session:
        yield session

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import time
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from sqlmodel import Field, Session, SQLModel, create_engine

engine = create_engine("postgresql+psycopg://postgres:postgres@localhost/db")


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str


app = FastAPI()


def get_session():
    with Session(engine) as session:
        yield session


def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")


def generate_stream(query: str):
    for ch in query:
        yield ch
        time.sleep(0.1)


@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

...would be run after the response finishes sending the slow data:

Python 3.10+

```python
# Code above omitted 👆

def generate_stream(query: str):
    for ch in query:
        yield ch
        time.sleep(0.1)


@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

👀 **Full file preview**

Python 3.10+

```python
import time
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from sqlmodel import Field, Session, SQLModel, create_engine

engine = create_engine("postgresql+psycopg://postgres:postgres@localhost/db")


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str


app = FastAPI()


def get_session():
    with Session(engine) as session:
        yield session


def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")


def generate_stream(query: str):
    for ch in query:
        yield ch
        time.sleep(0.1)


@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

But as `generate_stream()` doesn't use the database session, it is not really necessary to keep the session open while sending the response.

If you have this specific use case using SQLModel (or SQLAlchemy), you could explicitly close the session after you don't need it anymore:

Python 3.10+

```python
# Code above omitted 👆

def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")
    session.close()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
import time
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from sqlmodel import Field, Session, SQLModel, create_engine

engine = create_engine("postgresql+psycopg://postgres:postgres@localhost/db")


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str


app = FastAPI()


def get_session():
    with Session(engine) as session:
        yield session


def get_user(user_id: int, session: Annotated[Session, Depends(get_session)]):
    user = session.get(User, user_id)
    if not user:
        raise HTTPException(status_code=403, detail="Not authorized")
    session.close()


def generate_stream(query: str):
    for ch in query:
        yield ch
        time.sleep(0.1)


@app.get("/generate", dependencies=[Depends(get_user)])
def generate(query: str):
    return StreamingResponse(content=generate_stream(query))
```

That way the session would release the database connection, so other requests could use it.

If you have a different use case that needs to exit early from a dependency with `yield`, please create a GitHub Discussion Question with your specific use case and why you would benefit from having early closing for dependencies with `yield`.

If there are compelling use cases for early closing in dependencies with `yield`, I would consider adding a new way to opt in to early closing.

Dependencies with `yield` and `except`, Technical Details

Before FastAPI `0.110.0`, if you used a dependency with `yield`, and then you captured an exception with `except` in that dependency, and you didn't raise the exception again, the exception would be automatically raised/forwarded to any exception handlers or the internal server error handler.

This was changed in version `0.110.0` to fix unhandled memory consumption from forwarded exceptions without a handler (internal server errors), and to make it consistent with the behavior of regular Python code.

Background Tasks and Dependencies with `yield`, Technical Details

Before FastAPI `0.106.0`, raising exceptions after `yield` was not possible, the exit code in dependencies with `yield` was executed *after* the response was sent, so Exception Handlers would have already run.

This was designed this way mainly to allow using the same objects "yielded" by dependencies inside of background tasks, because the exit code would be executed after the background tasks were finished.

This was changed in FastAPI `0.106.0` with the intention to not hold resources while waiting for the response to travel through the network.

Tip

Additionally, a background task is normally an independent set of logic that should be handled separately, with its own resources (e.g. its own database connection).

So, this way you will probably have cleaner code.

If you used to rely on this behavior, now you should create the resources for background tasks inside the background task itself, and use internally only data that doesn't depend on the resources of dependencies with `yield`.

For example, instead of using the same database session, you would create a new database session inside of the background task, and you would obtain the objects from the database using this new session. And then instead of passing the object from the database as a parameter to the background task function, you would pass the ID of that object and then obtain the object again inside the background task function.